

M5cet 3.0

Šifrovaná komunikace peer-to-peer v prohlížeči: zprávy, soubory, hovory a sdílení polohy mezi lidmi, kteří znají společné heslo místnosti. Server zprostředkuje spojení, podrží zprávy pro nepřítomné a nikdy nevidí obsah.

Tato dokumentace popisuje verzi **3.0.0**: nový signalizační protokol v2, šifrování v2 s identitou zařízení, spolehlivou frontu zpráv, novou administraci s živým sledováním provozu a auditem.

Node.js 24

TypeScript 7

React 19

WebRTC DataChannel

AES-GCM 256

ECDSA P-256

WebAuthn + PRF

SQLite + SQLCipher

Pro koho je co

Uživatel — režimy, co šifrování chrání. **Vývojář** — architektura, protokol v2, API. **Provozovatel** — nasazení, administrace, audit, bezpečnost.

Co je nového ve 3.0

Validované rámce a limity, pseudonymy účtů, zapečetěná signalizace (ochrana hovorů před serverem), podepsané zprávy a soubory, fronta s lease a dead-letter, konzole s živým provozem a auditem. Plný přehled v seznamu změn.

Vyhledávání: stiskněte `/` a pište — výsledky se hledají v nadpisech i textu, `↑` `↓` a `Enter` skočí na místo. Tlačítko **PDF** stáhne tutéž dokumentaci k tisku.

Obsah

1. Úvod
2. Režimy a funkce
3. Architektura
4. Signalizační protokol v2
5. Šifrování v2
6. Fronta zpráv pro nepřítomné
7. Účty a passkeys
8. Úložiště
9. Administrace
10. Monitoring a audit
11. API reference
12. Nasazení
13. Bezpečnostní opatření serveru
14. Návrhy a roadmapa
15. Změny ve 3.0.0

Režimy a funkce

Aplikace běží ve dvou režimech. Oba šifrují stejně; liší se tím, co smí dělat server.

REŽIM	CO DĚLÁ SERVER	CO TO PŘINÁŠÍ
light	Jen signalizace (spojení WebRTC). Nic neukládá.	Nejmenší stopa. Zpráva pro někoho, kdo není online, čeká v lokální frontě prohlížeče.
server-enhanced	Navíc: úložiště, relay pro nepřítomné, Web Push, synchronizace nastavení, proxy souborů, když P2P nejde.	Zprávy dorazí i nepřítomnému (přihlášenému passkey), historie přežije zavření okna, soubory projdou i přes striktní NAT.

Uchování chatu

VOLBA	KDE JE HISTORIE	KDO JI PŘEČTE
ephemeral	Nikde. Nové spojení začíná prázdné.	—
session	V sessionStorage prohlížeče, šifrovaná neexportovatelným klíčem z IndexedDB.	Jen tato záložka prohlížeče.
server s passkey	V trezoru účtu a v jeho SQLCipher databázi; zapečetěno v prohlížeči klíčem z PRF passkey.	Jen držitel passkey (zero-knowledge).
server bez passkey	V dočasné databázi relace (TTL 1 den); každý řádek zapečetěný v prohlížeči klíčem, který zařízení neopustí.	Jen tento prohlížeč. Server drží řádky, které neumí otevřít.

Funkce v kostce

Zprávy

Místnost i soukromí příjemci, odpovědi, přeposílání, stav (odesláno → uloženo → doručeno → přečteno), zprávy *klikací*, *mizející* a *zapečetěné* vlastním kódem.

Soubory

Malé přímo ve zprávě, velké po kusech po DataChannelu; když P2P nejde, přes šifrovanou proxy serveru. Opakování ztracených kusů, ověření otisku celého souboru.

Hovory a média

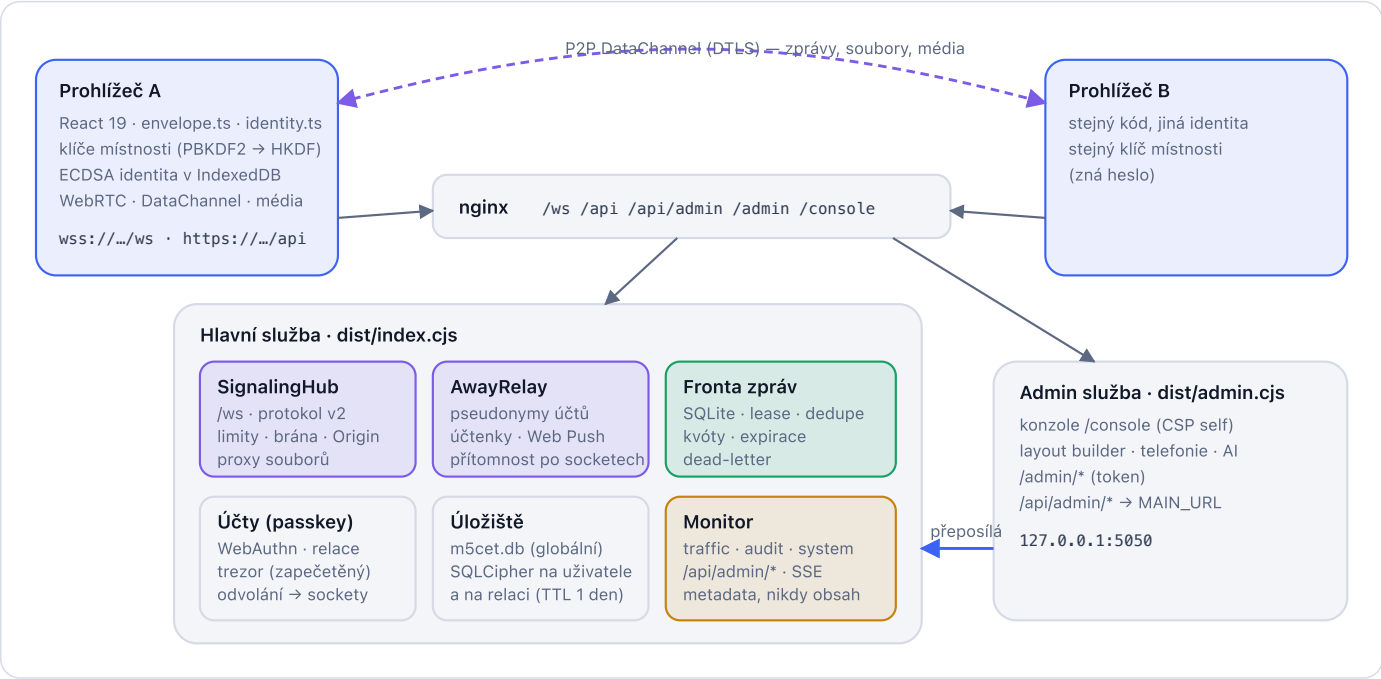
Audio a video přes WebRTC; DTLS otisky chráněné zapečetěnou signalizací. Sdílení polohy, NFC, přepis řeči, AI asistent a telefonie jako volitelné moduly.

Účty

Passkey (WebAuthn) s rozšířením PRF: z něj se v prohlížeči odvodí klíč trezoru i klíč vlastní databáze. Server nezná žádný z nich.

Architektura

Tři běžící části: **prohlížeče** (veškerá kryptografie), **hlavní služba** (Express + WebSocket, všechny živý stav) a **admin služba** (konzole a nástroje). Před nimi obvykle nginx.



Obr. 1 — Části systému a kudy tečou data. Fialové: signalizace a relay; zelené: fronta; oranžové: monitoring.

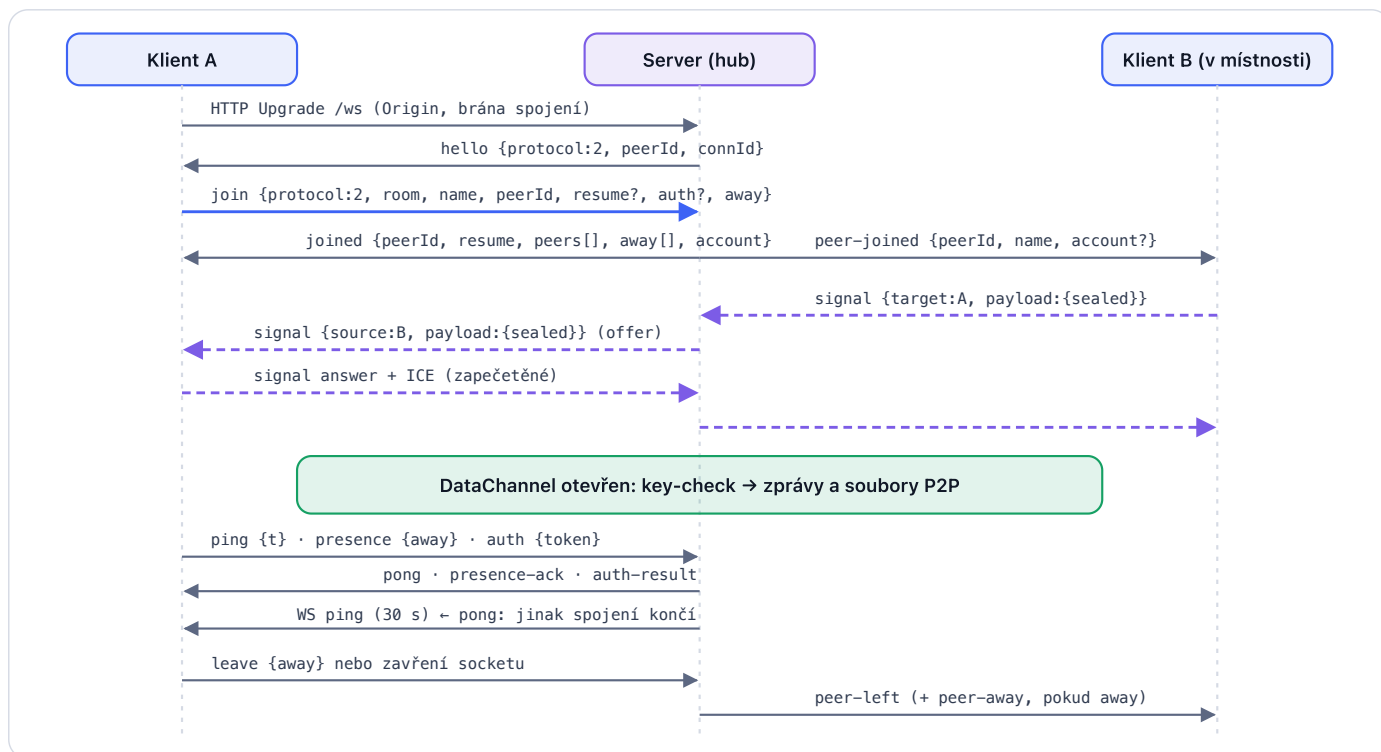
Kde je co v kódu

OBLAST	SOUBORY
Signalizace, protokol v2	server/signaling/hub.ts, frames.ts, limits.ts, refs.ts
Relay pro nepřítomné	server/signaling/relay.ts
Fronta zpráv	server/accounts/mailqueue.ts (SQLite), memqueue.ts (paměť)
Účty, WebAuthn	server/accounts/store.ts, routes.ts, webauthn.ts, cbor.ts
Úložiště	server/storage/* — service.ts, global-store.ts, user-store.ts, keys.ts
Monitoring a audit	server/monitor/traffic.ts, audit.ts, system.ts; API server/admin-api.ts
Admin služba a konzole	server/admin.ts, admin-ui/public/ (console.js, legacy-tools.js)
Kryptografie v prohlížeči	client/src/lib/envelope.ts, identity.ts, crypto.ts (v1), file-transfer.ts, message-kinds.ts
Validace přichozích dat	client/src/lib/validate.ts
Aplikace	client/src/App.tsx a komponenty v client/src/components/

Signalizační protokol v2

Jeden WebSocket na `/ws`, rámce jako JSON objekty s polem `type`. Verze 2 přinesla přesnou validaci, limity, pseudonymy, rámec `auth` a zapečetěnou signalizaci.

Životní cyklus spojení



Obr. 2 — Připojení, výměna zapečetěné signalizace a udržování spojení.

Co se stane s každým rámcem

- Velikost** — nad 256 KB (`MAX_FRAME_BYTES`) knihovna ws spojení zavře kódem `1009`.
- Parsování** (`parseFrame`) do přesného tvaru: typy, délky, formát ID (`[A-Za-z0-9_:-]{1,96}`), base64, počty. Neznámý typ → `error {code:"unknown-type"}`, špatný tvar → `invalid-frame`.
- Limit třídy** — token bucket na socket. Při překročení `rate-limited {frame, retryAfterMs}`; 20 odmítnutí za minutu → zavření `1008`.
- Akce a záznam** do monitoru provozu (typ, bajty, spojení, peer, účet, hash místnosti, stav).

Rámce klient → server

TYPE	POLE	VÝZNAM	TŘÍDA LIMITU
join	protocol, room, name, peerId?, resume?, auth?, away	Vstup do místnosti; <code>auth</code> = session token účtu	signaling
auth	token null, away	Přihlásit / odhlásit účet bez opuštění místnosti	signaling
leave	away	Odchod; <code>away:true</code> = zůstat jako nepřítomný (relay)	signaling
signal	target, payload	SDP/ICE pro jednoho peera; ve v2 <code>{sealed:{v,iv,ciphertext}}</code>	signaling
ping	t	Aplikační heartbeat a měření RTT	heartbeat
presence	away	Stránka odložena / zpět (lifecycle)	presence
relay	messageId, to[], envelope, expiresAt?	Šifrová obálka pro nepřítomné členy (odkazy z <code>away[]</code>)	relay
relay-ack	ids[] (≤ 500)	Potvrzení zpracování doručených položek	relay
receipt	messageIds[] (≤ 200), state	Přečteno / doručeno — jen pro zprávy, které server tomuto čtenáři doručil	receipt
storage	id, op, payload, auth?, session?	Operace úložiště po WebSocketu (viz úložiště)	storage
command-poll / command-ack	deviceId / commandId, result?	Příkazy operátora pro zařízení	command
proxy-meta chunk end cancel need	transferId, ...	Proxy souborů přes server (když P2P nejde)	proxy (+ bajty)

Rámce server → klient

TYPE	KDY
<code>hello</code>	Hned po otevření: <code>protocol</code> , navržené <code>peerId</code> , <code>connId</code> , <code>limits</code>
<code>joined</code>	Odpověď na <code>join</code> : přidělené <code>peerId</code> , <code>resume</code> , <code>peers[]</code> , <code>away[]</code> , <code>account</code>
<code>peer-joined</code> / <code>peer-left</code> / <code>peer-updated</code>	Změny v místnosti (<code>peer-updated</code> = přihlášení/odhlášení bez odchodu)
<code>peer-away</code> / <code>peer-back</code> / <code>peer-gone</code>	Člen s účtem je nepřítomen / zpět / odhlášen (relay pro něj končí)
<code>signal</code> , <code>signal-undeliverable</code>	Signál od peera / cíl v místnosti není
<code>relay-status</code>	<code>stored</code> <code>forwarded</code> <code>duplicate</code> <code>delivered</code> <code>read</code> <code>rejected</code> pro odeslanou zprávu
<code>relay-deliver</code>	Položky z fronty (po 50) — čekají na <code>relay-ack</code>
<code>presence-ack</code> , <code>auth-result</code> , <code>account-revoked</code>	Stav přítomnosti a účtu na tomto socketu
<code>rate-limited</code> , <code>error</code>	Odřmítnutý rámce (<code>code</code> : <code>invalid-frame</code> , <code>unknown-type</code> , <code>not-in-room</code> , <code>proxy-refused</code> , <code>server-error</code>)
<code>replaced</code> , <code>closed-by-server</code>	Stejný klient se připojil znovu (resume) / operátor spojení ukončil
<code>proxy-*</code> , <code>proxy-ack</code>	Přeposlané rámce proxy (s <code>from</code>) a přijetí přenosu
<code>admin-command</code> , <code>storage-result</code> , <code>pong</code>	Příkaz operátora, výsledek operace úložiště, odpověď na ping

Datagramy — rozložení klíčových rámců

☐ hlavička / směrování (vidí server) ☐ bezpečnostní pole ☐ šifrový text (server nepřečte)

join					klient → server
type	protocol	room		name	
"join"	2	1–64 znaků		≤ 48, bez řídících znaků	
peerId?		resume?		auth?	away
požadované ID		tajemství z minulého joined		session token (≤ 200)	bool

joined					server → klient
type	protocol	peerId		resume	
"joined"	2	přidělené — převzít!		32 B base64url, jednorázově	
peers[]		away[]		account	
{peerId, name, joinedAt, account?}		{account, name, since}		{account, away} {invalid} null	

signal (zapečetěný)					klient → server → klient
type	target / source	sealed.v	sealed.iv	sealed.ciphertext	
"signal"	peer ID	2	12 B	AES-GCM(klíč „signal“, SDP ICE) · AAD „m5cet/2 signal místnost od pro“	

relay					klient → server
type	messageId		to[]		expiresAt?
"relay"	ID zprávy		≤ 50 odkazů účtů (pseudonymy)		TTL zprávy
envelope.v	envelope.id	envelope.iv	envelope.ciphertext		
2	= messageId	12 B	AES-GCM(klíč „message“, {b, pk?, s?}) · AAD „m5cet/2 msg místnost id“ · ≤ 190 KB		

relay-deliver — položka						server → klient	
id mq-... (pro ack)	seq pořadí v místnosti	kind message status	messageId		from {peerId, name, account?}		
envelope přesně to, co odesílatel poslal			status? {state, at, recipientName}		storedAt	attempts	

proxy-chunk							odesílatel → server → místnost	
type "proxy-chunk"	transferId	seq 0 ... n-1	v 2	iv	ciphertext AES-GCM(klíč souboru, kus) · AAD „m5cet/2 chunk přenos seq počet“		from přidá server	

Limity

TŘÍDA	KAPACITA (BURST)	DOPLŇOVÁNÍ / S	RÁMCE
signaling	120	10	join, leave, signal, auth
relay	60	2	relay, relay-ack
receipt	30	1	receipt
presence	10	0,2	presence
storage	120	10	storage
proxy	400	60	proxy-* + bajty: 8 MB burst, 2 MB/s
heartbeat	6	0,5	ping
command	20	0,5	command-poll, command-ack
other	30	1	neplatné rámce

Brána spojení (při upgradu): 30 nových spojení za minutu a 20 současně na adresu, 5 000 celkem. Adresa se bere z `X-Forwarded-For` jen od důvěryhodné proxy (`TRUST_PROXY`). **Zpětný tlak**: příjemci s více než 8 MB ve frontě socketu server přestane posílat kusy souborů (klient si je vyžádá znovu), nad 32 MB je spojení ukončeno.

Peer ID, resume a pseudonymy

- Server v `hello` navrhne ID; klient smí v `join` požádat o vlastní. Je-li v místnosti obsazené jiným živým socketem, dostane nové — **převzetí cizího ID už není možné** (dřív šlo odklonit cizí signalizaci).
- `joined.resume` je jednorázové tajemství. Při novém připojení do stejné místnosti ho klient pošle spolu se svým ID; server porovná SHA-256 v konstantním čase, starý socket dostane `replaced` a ID přejde na nové spojení.
- Členové s účtem se místnosti ukazují jako `account = (base64url(HMAC-SHA256(tajemství, místnost || 0 || ID účtu)) [0..22])`. Stejná osoba má v každé místnosti jiný odkaz. Pole `accountId` nese totéž (alias pro klienty 2.x).

Příklad: minimální klient

```
const ws = new WebSocket("wss://chat.example.com/ws");
let me = null;
ws.onmessage = (e) => {
  const f = JSON.parse(e.data);
  if (f.type === "hello") ws.send(JSON.stringify({ type: "join", protocol: 2, room: "brno", name: "Alice", peerId: f.peerId, away: false }));
  if (f.type === "joined") { me = f.peerId; /* uložit f.resume pro reconnect */ }
  if (f.type === "rate-limited") setTimeout(retry, f.retryAfterMs);
  if (f.type === "error") console.warn(f.code, f.message);
};
setInterval(() => ws.send(JSON.stringify({ type: "ping", t: Date.now() })), 25_000);
```

Kompatibilita: klient 3.0 přijímá jen zapečetěné signály. Klient 2.x se po nasazení sám nabídne k obnovení; dokud se neobnoví, P2P spojení s ním nevznikne a jeho zprávy v1 klient 3.0 sice přečte, ale 2.x nepřečte zprávy v2.

Šifrování v2

Veškerá kryptografie běží v prohlížeči přes WebCrypto. Server dostává jen šifrované texty a metadata potřebná ke směrování.

Hierarchie klíčů místnosti



Obr. 3 — Z jednoho hesla čtyři nezávislé klíče; žádný z nich neopustí prohlížeč.

Svázání s kontextem (AAD)

Každé šifrování AES-GCM nese *associated data* — řetězec, který se nešifruje, ale je chráněný tagem. Šifrový text proto jde otevřít jen tam, kam patří.

ÚČEL	AAD	ČEMU BRÁNÍ
zpráva	<code>m5cet/2 msg <místnost> <id></code>	přenesení do jiné místnosti, záměně ID, přehrání pod jiným ID
signál	<code>m5cet/2 signal <místnost> <od> <pro></code>	přesměrování nebo podvržení SDP serverem (MITM hovorů)
meta souboru	<code>m5cet/2 file-meta <přenos></code>	záměně metadat mezi přenosy
kus souboru	<code>m5cet/2 chunk <přenos> <seq> <počet></code>	přeskládání kusů, vložení kusu z jiného souboru
konec souboru	<code>m5cet/2 file-end <přenos></code>	podvržení otisku
řádek historie na serveru	<code>m5cet:server-row:v1:<id></code>	záměně řádků v úložišti relace

Obálka zprávy

Envelope v2				DataChannel i relay
v	id	iv	ciphertext	
2	ID zprávy (v AAD)	12 B náhodně	AES-GCM-256(JSON { b: tělo, pk?: veřejný klíč, s?: podpis }) + 16 B tag	
tělo b (uvnitř šifrování)				jen příjemci
id	text / attachment	senderId, senderName	createdAt, ttl, flags	to, replyTo, forwardedFrom
= envelope.id	obsah, příloha data:			

Podpis `s` = ECDSA-P256-SHA256 přes `AAD || b` klíčem identity odesílatele; `pk` je jeho veřejný klíč (SPKI). Obojí je **uvnitř** šifrování — server nevidí, kdo co podepsal, a nemůže tak propojit identitu zařízení napříč místnostmi.

```
// client/src/lib/envelope.ts
const keys = await deriveRoomKeys("brno", "correct horse battery");
const me = await loadIdentity(); // ECDSA P-256, IndexedDB
const env = await sealMessage(keys, "msg-1", { id: "msg-1", text: "ahoj", /* ... */ }, me);
// → { v: 2, id: "msg-1", iv: "...", ciphertext: "..." }
const { payload, version, signer } = await openMessage(keys, env);
// signer = { publicKey, valid: true } · version = 2
```

Identita zařízení, TOFU a bezpečnostní čísla

- Každý profil prohlížeče má pár ECDSA P-256; soukromý klíč je **neexportovatelný** (umí podepsat, nejde přechít ani kódem aplikace) a leží v IndexedDB `m5cet-identity`.
- **Trust on first use:** první ověřený klíč pro jméno v místnosti se připne (`m5cet:pins:v1`). Zpráva pod stejným jménem s jiným klíčem se označí **jiný klíč** a zobrazí se varování; neplatný podpis **neplatný**; starší klient **nepodepsáno**.
- **Bezpečnostní číslo** dvou lidí = 12 skupin po 5 číslicích ze SHA-512 (1 025 kol) seřazených veřejných klíčů — obě strany vidí totéž a mohou ho porovnat mimo aplikaci.
- Otisk vlastního zařízení i ověření odesílatele ukazuje *Informace o zprávě*.

Kontrola klíče a replay

Po otevření DataChannelu si peeri pošlou `{kind:"key-check", v:2, check}`. Nesouhlasí-li, aplikace řekne „jiné heslo místnosti“ místo nečitelných zpráv. Hodnota je odvozená z tajemství místnosti a k útoku na heslo nedává víc než libovolný šifrový text. ID přijatých zpráv si klient pamatuje (20 000 posledních) a opakovaně zahodí.

Soubory

1. Odesílatel vytvoří `transferId`, odvodí **klíč souboru** (HKDF, salt = ID přenosu) a pošle zapečetěná, podepsaná metadata.
2. Každý kus (32 KiB) šifruje s AAD `chunk|přenos|seq|počet` a zároveň počítá jeho SHA-256.
3. Konec přenosu nese zapečetěné a podepsané `{ root, totalChunks, size }`, kde `root` = SHA-256 zřetězených otisků kusů.
4. Příjemce ověří: metadata před alokací (velikost, počet kusů ≤ 2 000 000, konzistence), každý kus proti AAD, na konci součet bajtů, `root` a shodu podpisu s metadaty. Ztracené kusy si vyžádá (`file-need` / `proxy-need`, max. 3 kola).
5. Jméno souboru se očistí (bez cest a řídicích znaků, BiDi) a **MIME typ se omezí na bezpečné** (obrázky, audio, video, PDF, text). HTML nebo SVG přijde jako `application/octet-stream` — nikdy se nespustí jako stránka v originu aplikace.

Zapečetěné zprávy

Text zprávy se navíc šifruje kódem předaným mimo aplikaci. Ve 3.0: kód **12 znaků** z 31znakové abecedy bez záměnných znaků (≈ 59 bitů, zobrazení `XXXX-XXXX-XXXX`), losování bez modulo biasu, PBKDF2 **600 000** iterací, při zadání se ignoruje velikost písmen, mezery a pomlčky. Zprávy z 2.x (6 znaků, 150 000 iterací) jdou dál otevřít. Text ani kód zapečetěné zprávy se nikdy neukládají mimo prohlížeč.

Co kdo vidí

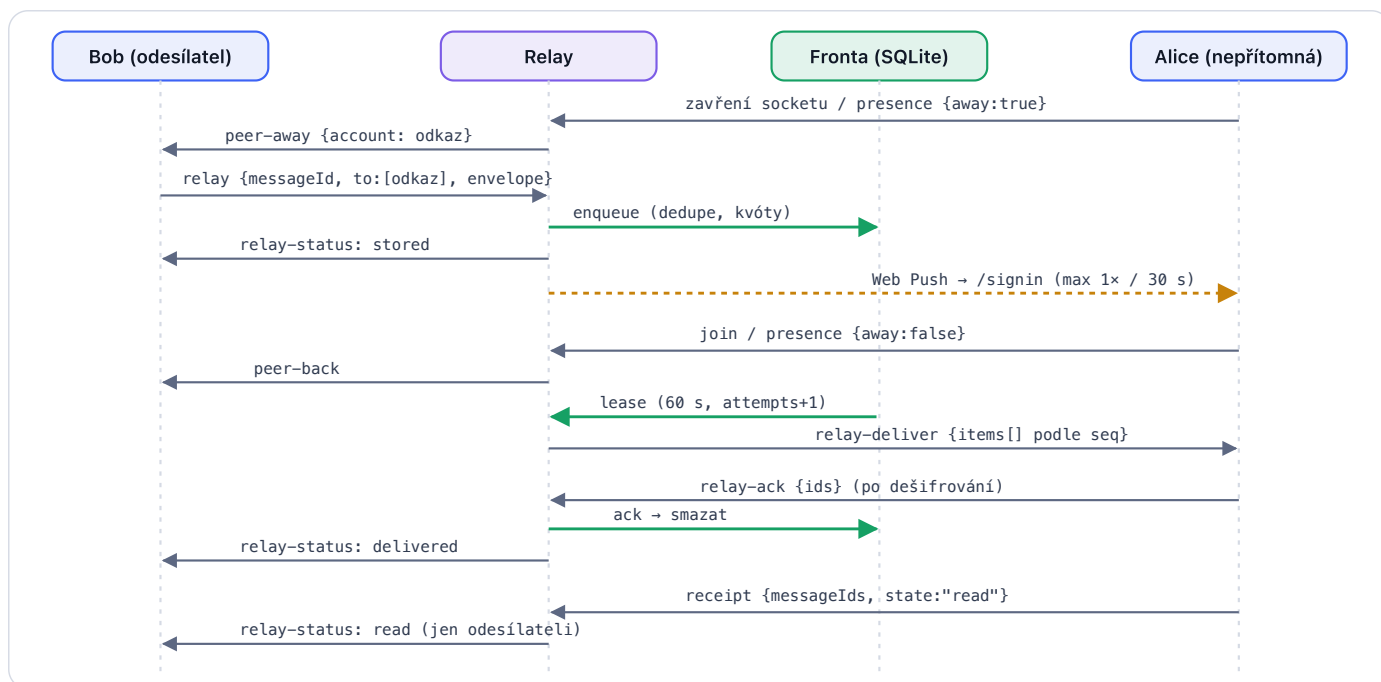
ÚDAJ	SERVER	ČLEN MÍSTNOSTI	POZOROVATEL SÍTĚ
Obsah zpráv, souborů, hovorů	ne	ano (má klíč)	ne
Kdo zprávu napsal (podpis)	ne	ano, ověřeně	ne
SDP / ICE, DTLS otisky	ne (zapečetěno)	ano	ne
Název místnosti	ano (směrování)	ano	ne (TLS)
Kdo je v místnosti, kdy, IP	ano (monitor ukládá IP zkrácenou)	jména	IP spojení
Kdo komu píše přes relay	ano — do auditu jen se zapnutým auditem komunikace	—	ne
Heslo místnosti, klíče, passkey	nikdy	heslo ano	ne

Omezení, které 3.0 neřeší: klíč místnosti je sdílený a statický — kdo heslo zná (nebo ho později získá), přečte i starší zachycený provoz. Forward secrecy a vyloučení člena bez změny hesla řeší až návrhy (sender keys / MLS).

Fronta zpráv pro nepřítomné

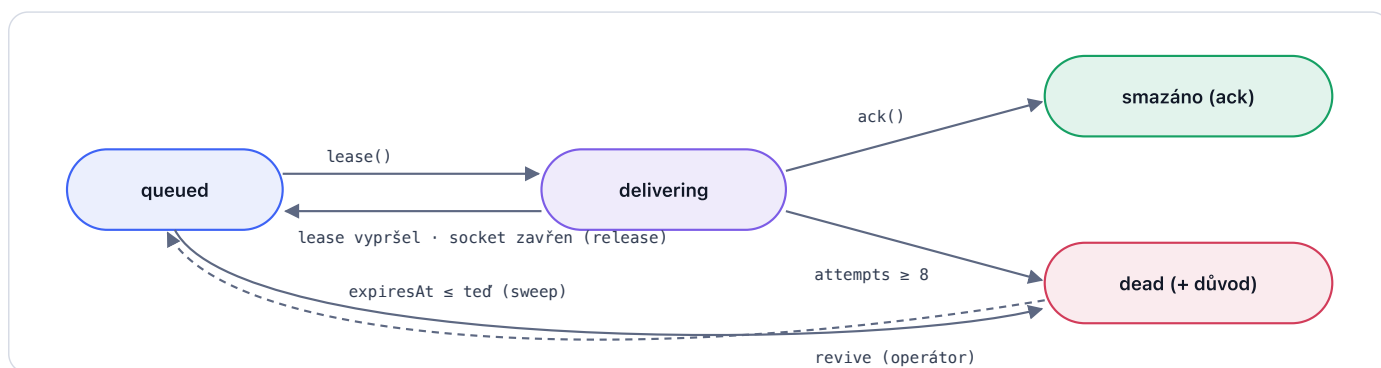
Když člen s účtem (passkey) a zapnutým uchováním na serveru odejde nebo odloží stránku, zůstane v místnosti jako *nepřítomný*. Ostatní mu dál píší; server šifrovou obálku uloží, odpoví „uloženo“, probudí ho Web Push a po návratu mu vše předá — v pořadí a s potvrzením.

Tok relay



Obr. 4 — Od odeslání nepřítomné po „přečteno“. Je-li odesílatel mezitím pryč, stav čeká v jeho vlastní frontě jako položka `status`.

Stavy položky



Obr. 5 — Doručení položku nemaže; smaže ji až potvrzení příjemce.

VLASTNOST	HODNOTA	POZNÁMKA
Pořadí	<code>seq</code> na (účet, místnost)	doručení v pořadí odeslání
Deduplikace	(účet, druh, messageId)	opakovaný relay = <code>duplicate</code>
Položek na účet	1 000	<code>maxItemsPerAccount</code>
Bajtů na účet	8 MB	<code>maxBytesPerAccount</code>
Velikost položky	130 000 B	<code>maxItemBytes</code>
Od jednoho odesílatele	300 položek	jeden člen nezaplní cizí schránku
Lease	60 s	po zavření socketu uvolněn hned
Pokusů	8	pak <code>dead</code>
Život	30 dní (<code>RELAY_RETENTION_DAYS</code>) nebo TTL zprávy	kratší z obou
Dead-letter	7 dní	pak se maže

Poctivé účtenky

Server si ke každé relayované zprávě pamatuje (v paměti, až 50 000 zpráv / 30 dní) odesílatele a příjemce. `receipt` přijme jen od účtu, kterému zprávu doručil, a pošle ho jen jejímu odesílateli. Ve 2.x mohl kdokoli v místnosti označit cizí zprávu jako přečtenou a vložit položky do cizí schránky.

Lokální fronta (light režim)

Bez serveru čeká zpráva, pro kterou není otevřený kanál, v paměti prohlížeče — už zašifrovaná. Zkouší se při otevření kanálu, po návratu do okna a jednou za minutu (200 zpráv, 60 pokusů, 24 h, respektuje TTL). Protože peer ID je od 3.0 stabilní i po výpadku (resume), obálky ve frontě zůstávají platné.

Účty a passkeys

Účet je passkey (WebAuthn) — žádné heslo. Server uloží jen veřejný klíč a počítaadlo podpisů; bez rozšíření **PRF** účet nevznikne, protože z něj prohlížeč odvozuje klíče:

- `HKDF(PRF, "m5cet:profile:v1")` → klíč trezoru (profil + historie chatu, zapečetěné před odesláním),
- `HKDF(PRF, "m5cet:userdb:v1")` → klíč vlastní SQLCipher databáze; server ho drží jen v paměti, dokud je uživatel přihlášený.

MECHANISMUS	CHOVÁNÍ
Výzvy	náhodné, jednorázové, 2 minuty, vázané na obřad (registrace / přihlášení)
Relace	náhodný token (32 B), na serveru jen SHA-256, platnost 12 h, jen v paměti — restart znamená nové přihlášení (klient to udělá sám)
Odvolání	odhlášení (jedno zařízení), „všude“, smazání účtu, zásah operátora → <code>account-revoked</code> na všech dotčených socketech, relay pro účet končí, databáze se pro toto zařízení zamkne (poslední držitel ji zavře úplně)
RP ID	<code>WEBAUTHN_RP_ID</code> , jinak host z <code>PUBLIC_BASE_URL</code> , jinak hlavička Host (v produkci server varuje)
Push	jen endpointy známých služeb (FCM, Mozilla, Windows, Apple; rozšíření <code>PUSH_ENDPOINT_HOSTS</code>)

Úložiště

Jedna globální SQLite databáze pro server a jedna SQLCipher databáze pro každého uživatele či anonymní relaci. Podrobně v `docs/storage.md`.

```
$DATA_DIR/storage/
├─ m5cet.db      # globální: uživatelé, passkeys, index DB, logy, přenosy, audit, fronta
├─ storage.key   # master klíč, 0600 (nebo STORAGE_MASTER_KEY / STORAGE_KEY_FILE)
└─ db/
   ├─ db-<id>.db # účet – klíč z passkey (PRF), jen v paměti
   └─ db-<id>.db # relace – klíč zabalený master klíčem, TTL 24 h od posledního použití, max. 7 dní
```

Limity

Kvóta 64 MB na účet, 16 MB na relaci (zápis nad kvótou se vrátí); zpráva ≤ 256 KB; hodnota kv ≤ 4 MB; 30 nových relací za hodinu z jedné adresy, 5 000 živých celkem; nejvýš 200 otevřených databází (LRU).

Povýšení relace na účet

Po vytvoření passkey se data relace zkopírují do databáze účtu v jedné transakci (ATTACH s klíčem), počty se ověří před commitem; relace se smaže až potom.

Klíče

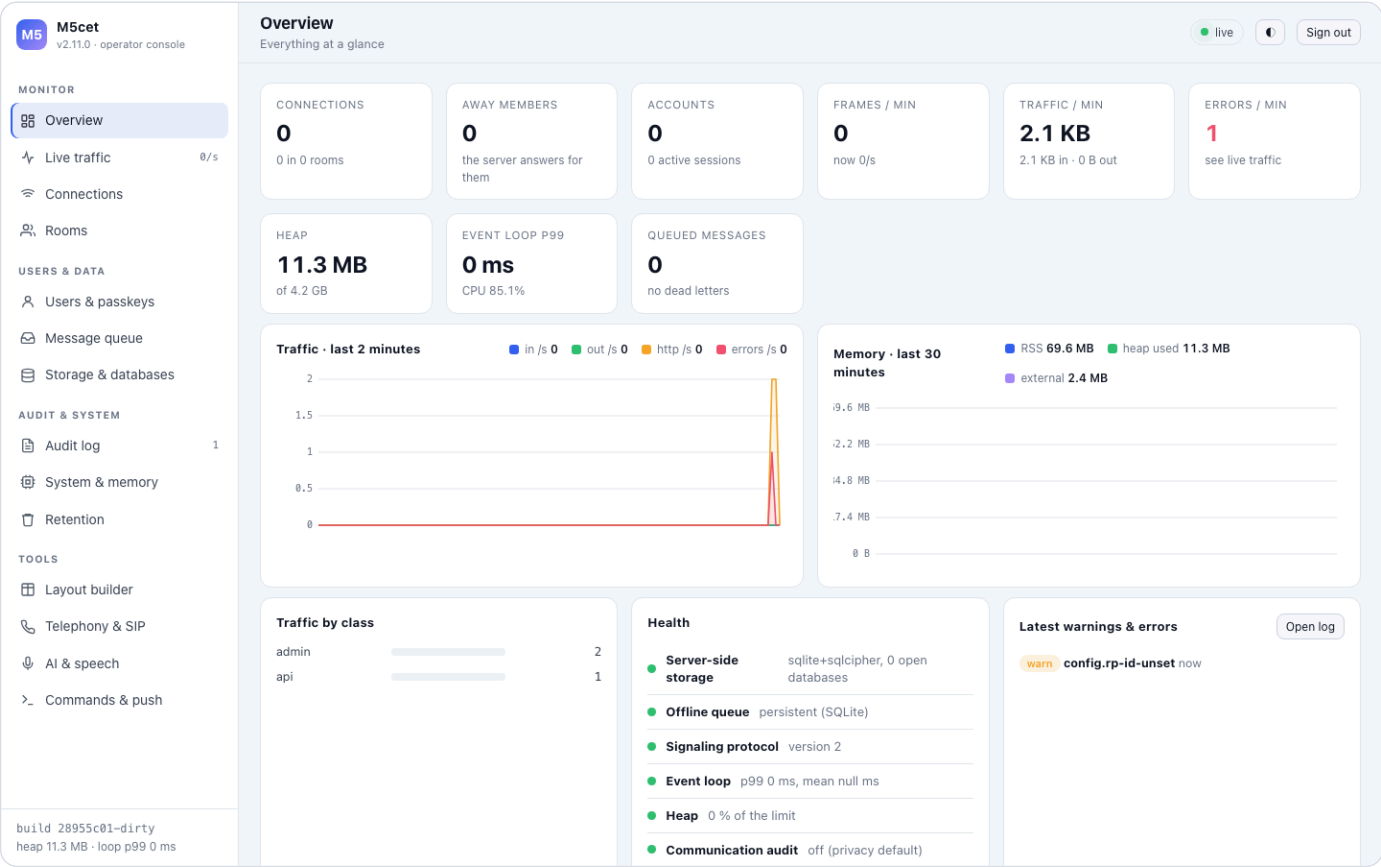
Klíč účtu drží server po držitelích (zařízeních); zapomene ho a přepíše nulami, až se odhlásí poslední, nebo po 12 h nečinnosti. ID relací jen jako HMAC.

Veřejný status

`/api/storage/status` říká jen `available`, `engine`, `caller`, `locked` — nic o ostatních uživateli. Úplný stav je v administraci.

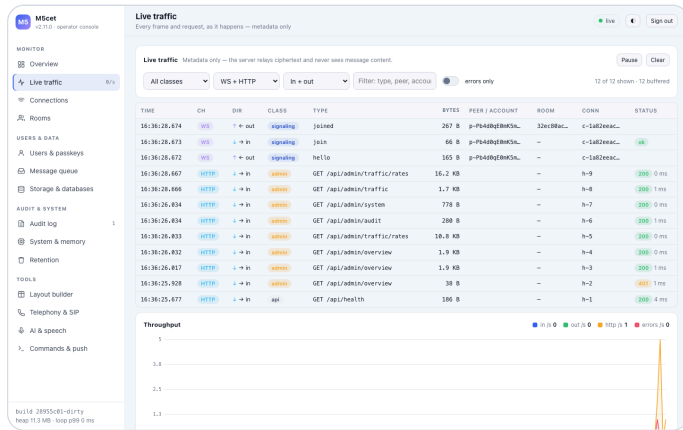
Administrace

Konzole operátora na `https://<doména>/console/`. Živá data bere z hlavní služby (`/api/admin/*`), nástroje layoutu, telefonie a AI z admin služby (`/admin/*`). Vše chrání `ADMIN_API_TOKEN`.

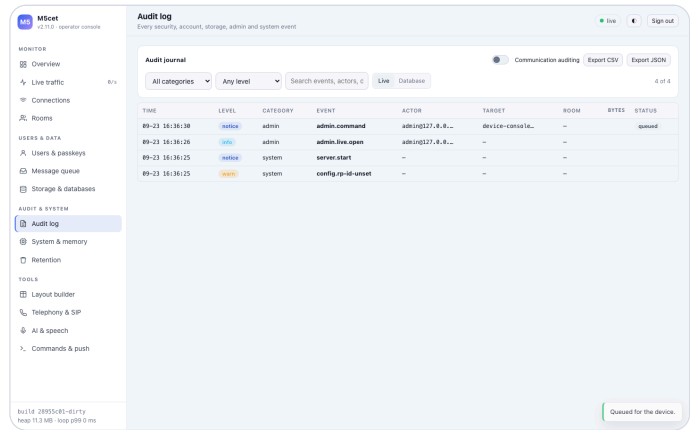


Obr. 6 — Přehled: KPI, provoz za 2 minuty, paměť za 30 minut, provoz podle tříd, zdraví, poslední varování.

PANEL	CO UKAZUJE A CO UMÍ
Přehled	spojení, místnosti, nepřítomní, účty a relace, rámce a bajty za minutu, chyby, heap, event loop p99, fronta; grafy; zdravotní kontroly (úložiště, trvalost fronty, event loop, heap, audit komunikace)
Živý provoz	každý WebSocket rámec a HTTP požadavek (čas, kanál, směr, třída, typ, bajty, peer/účet, hash místnosti, spojení, stav, doba); filtry, pauza, detail; graf propustnosti
Spojení	otevřené sockety: jméno, peer, účet, místnost, klient (prohlížeč/OS), adresa /24 (/48), bajty a rámce, doba, protokol; odpojení
Místnosti	členové online i nepřítomní (hash místnosti, nikdy název)
Uživatelé a passkeys	účty, databáze (klíčování, otevřená), trezor, fronta, relace; detail s passkeys, položkami ve frontě, auditem účtu, žurnálem a provozem; odhlásit všude, smazat (s opsáním ID)
Fronta zpráv	stav fronty (uloženo / doručuje se / mrtvé / velikost / trvalost), schránky po účtech, dead-letter s obnovou
Úložiště	globální DB (soubor, WAL, stránky, žurnál, tabulky a řádky), index šifrovaných databází
Audit	žurnál s filtry (kategorie, úroveň, text), zdroj paměť / databáze, export CSV a JSON, přepínač auditu komunikace
System a paměť	RSS, heap (a limit), external, event loop p99 / průměr, CPU, zátěž hostitele, otevřené zdroje; grafy 30 minut
Retence	politika uchovávání, další běh, ruční spuštění
Příkazy a push	příkaz pro zařízení (doručí se hned, je-li připojené), čekající příkazy, log doručení, stav VAPID, odběratelé, testovací push
Layout, Telefonie, AI	přenesené nástroje z předchozí administrace



Obr. 7 — Živý provoz.



Obr. 8 — Auditní žurnál.

Bezpečnost konzole

- Token je jen v paměti záložky (volitelně v `sessionStorage`), nikdy v `localStorage` — starý klíč `m5cet:admin:cfg` konzole při načtení smaže.
- CSP `script-src 'self'`, žádné inline skripty ani `eval`, `frame-ancestors 'none'`. Vše ze serveru jde do stránky přes `textContent`.
- Živý stream je Server-Sent Events čtený přes `fetch()` — token v hlavičce, ne v URL (nezůstane v logu proxy).
- Odmítnuté tokeny se počítají: 30 za 15 minut na adresu. Každý zásah operátora se zapíše do auditu (kategorie `admin`).
- Doporučeno omezit `/console/`, `/admin/` a `/api/admin/` v nginx na adresy správců (`allow ... ; deny all;`).

Monitoring a audit

Záznam provozu

Monitor drží v paměti posledních 5 000 záznamů, počítadla po třídách, sekundové řady za 10 minut a registr živých spojení. Obsah zpráv nemá — server ho nikdy nevidí.

TrafficRecord							SSE event „traffic“
id	at ms	channel ws http	direction in out	cls třída	type „join“, „GET /api/..“		bytes
conn	peerId	accountId	roomHash SHA-256, 16 hex	ip /24 · /48	status ok · error · dropped · HTTP		durationMs

Třídy: `signaling`, `presence`, `relay`, `storage`, `file-proxy`, `heartbeat`, `account`, `admin`, `push`, `api`, `static`, `error`, `other`.
Heartbeaty se do živého streamu nepošílají (jejich počet je v ticku).

Auditní žurnál

KATEGORIE	PŘÍKLADY UDÁLOSTÍ	ZAPNUTO
<code>security</code>	<code>ws.origin-refused</code> , <code>ws.kicked</code> , <code>auth.invalid-token</code> , <code>join.peer-id-taken</code> , <code>relay.refused</code> , <code>http.rate-limited</code>	vždy
<code>account</code>	<code>account.register</code> , <code>account.sign-in</code> , <code>account.deleted</code> , <code>session.revoked</code> , <code>relay.away</code> , <code>relay.back</code> , <code>relay.push</code>	vždy
<code>communication</code>	<code>room.join</code> , <code>room.leave</code> , <code>relay.stored</code> , <code>relay.delivered</code>	jen na vyžádání (<code>AUDIT_COMMUNICATION=1</code> nebo přepínač)
<code>storage</code>	<code>queue.sweep</code> , zakládání, povýšení a mazání databází	vždy
<code>admin</code>	<code>admin.command</code> , <code>admin.user.delete</code> , <code>admin.audit.export</code> , <code>admin.connection.close</code>	vždy
<code>network</code>	<code>ws.refused</code> , <code>ws.closed</code> , <code>ws.heartbeat-timeout</code> , <code>ws.slow-consumer</code>	vždy
<code>system</code>	<code>server.start</code> , <code>server.stop</code> , <code>http.error</code> , <code>config.rp-id-unset</code> , <code>queue.migrated</code>	vždy

Úrovně `debug` < `info` < `notice` < `warn` < `error`. Žurnál drží 3 000 posledních záznamů v paměti; s úložištěm jde každý záznam i do tabulky `audit` globální databáze, kde je volné pole `detail` **zapečetěné** master klíčem a svázané s kategorií, událostí a časem.

Audit komunikace zaznamenává, kdo koho kdy a jak velkou zprávou oslovil — tedy přesně metadata, která by šifrovaný chat neměl sbírat. Proto je vypnutý a jeho zapnutí (i vypnutí) se samo zapíše do auditu. Zapínejte jen s právním důvodem.

Živý stream

```
GET /api/admin/live?streams=traffic,audit,tick
Authorization: Bearer <ADMIN_API_TOKEN>
Accept: text/event-stream

event: traffic
data: {"id":812,"at":1790000000000,"channel":"ws","direction":"in","cls":"relay","type":"relay","bytes":1843,"conn":"c-1a82ee...", "status":}

event: audit
data: {"id":77,"category":"security", "level":"warn", "event":"ws.kicked", "status":"rate limits exceeded"}

event: tick    (každé 2 s)
data: {"counts":{"..."},"rate":{"framesIn":12,"framesOut":15,"http":1,...},"system":{"rss":..., "loopP99":1.4}}
```

Konzoli, která nestíhá (víc než 1 MB ve frontě odpovědi), server záznamy zahazuje — nikdy kvůli ní nehromadí paměť.

API reference

Veřejné REST API hlavní služby

METODA A CESTA	POPIS	AUTORIZACE
GET /api/health	živost, verze, build, protocol, počet místností	—
GET /api/modules	manifest volitelných modulů	—
GET /api/turn	ICE servery (TURN, je-li nastaven)	—
GET /api/layout	vzhled a šablony od operátora	—
POST /api/share/create redeem revoke	pozvánky (split-key, kód 12 číslic); limit 30 / 40 za 15 min	—
GET /api/push/status, POST /api/push/subscribe test	Web Push pro zařízení bez účtu (jen známé push služby)	test sobě bez tokenu, všem s admin tokenem
/api/account/*	registrace a přihlášení passkey, me, trezor, push, odhlášení, smazání	Bearer token relace
/api/storage/*	úložiště (i přes WebSocket rámec storage)	relace účtu nebo X-M5cet-Session
GET POST /api/settings, POST /api/audit/purge, /api/analytics/consent	nastavení a souhlasy vázané na deviceId	znalost deviceId
/api/ai/*, /api/speech/*, /api/telephony/*	volitelné moduly (ENABLE_AI, ENABLE_SPEECH, ENABLE_TELEPHONY)	limity na IP
GET /api/events/recent, GET /api/transfers/stats	metadata událostí, statistika proxy	admin token (od 3.0)

Operátorské API (/api/admin/*)

Všechny cesty vyžadují Authorization: Bearer <ADMIN_API_TOKEN> .

METODA A CESTA	POPIS
GET overview	verze, proces, počty, provoz, fronta, úložiště, audit, zdraví
GET live	SSE: traffic, audit, tick (výběr ?streams=)
GET traffic	záznamy provozu; filtry cls, channel, direction, conn, peer, account, room, type, since, before, errors=1, limit
GET traffic/rates?seconds=	sekundové řady (max 600 s)
GET connections, POST connections/:id/close	živá spojení, odpojení
GET rooms	místnosti (hash), členové, nepřítomní
GET users, GET users/:id	účty; detail s passkeys, frontou, auditem, žurnálem, provozem
POST users/:id/signout	odvolat všechny relace (okamžitě i na socketech)
DELETE users/:id?confirm=:id	smazat účet se vším (ID se musí zopakovat)
GET queue, GET queue/dead, POST queue/:id/revive	fronta po účtech, dead-letter, obnova položky
GET system, GET db	proces a paměť (+ historie 30 min); databáze
GET audit (?source=db), GET audit/export?format=csv json	žurnál z paměti nebo databáze; export (CSV s ochranou proti vzorcům)
PUT audit/settings	{ communication: bool }
GET POST commands	povolené příkazy, čekající, log; zařazení příkazu {kind, deviceId, payload?}
GET push, POST push/test	stav VAPID a odběratelé; testovací push
GET events	metadata událostí (LOG_EVENTS=1)
GET retention, POST retention/run	politika uchovávání, ruční běh
GET storage, GET storage/logs	index a statistika úložiště, tabulka logů

```
# export auditu bezpečnostních událostí od úrovně warn do CSV
curl -H "Authorization: Bearer $ADMIN_API_TOKEN" \
  "https://chat.example.com/api/admin/audit/export?format=csv&category=security&minLevel=warn" -o audit.csv

# příkaz pro zařízení (doručí se hned, je-li připojené)
curl -X POST -H "Authorization: Bearer $ADMIN_API_TOKEN" -H "Content-Type: application/json" \
  -d '{"kind":"show-notification","deviceId":"dev-3f9c0a","payload":{"title":"Údržba","body":"Ve 22:00 restart"}}' \
  https://chat.example.com/api/admin/commands
```

Admin služba (/admin/*)

Layout builder (/admin/layout), telefonie a SIP (/admin/telephony/* včetně sip/trunks), AI a řeč (/admin/plugins*, /admin/logs/stream). Dřívější /admin/commands/*, /admin/clients, /admin/test/push, /admin/metrics a /admin/logs/recent dnes přeposílají na hlavní službu — ve 2.x četly prázdnou kopii stavu v jiném procesu, takže příkazy nikdy nedorazily.

Nasazení

Proměnné prostředí

PROMĚNNÁ	VÝCHOZÍ	VÝZNAM
<code>PORT</code> , <code>HOST</code>	5000, 0.0.0.0	kde poslouchá hlavní služba
<code>DATA_DIR</code>	<code>./m5cet</code>	data: účty, úložiště, <code>signaling.secret</code> (systemd: <code>StateDirectory</code>)
<code>TRUST_PROXY</code>	loopback	kterým proxy věřit s <code>X-Forwarded-For</code> (limity, brána spojení, audit)
<code>ALLOWED_ORIGINS</code> 3.0	—	další originy směřující otevřít <code>/ws</code> (čárkami); vlastní doména je povolená vždy
<code>ADMIN_API_TOKEN</code>	—	token konzole a admin API (bez něj vrací 503)
<code>ENABLE_ADMIN</code> , <code>ADMIN_PORT</code> , <code>ADMIN_BIND</code>	—, 5050, 127.0.0.1	admin služba
<code>MAIN_URL</code> 3.0	<code>http://127.0.0.1:\$PORT</code>	kam admin služba přeposílá <code>/api/admin/*</code>
<code>AUDIT_COMMUNICATION</code> 3.0	vypnuto	<code>1</code> = audit komunikace od startu
<code>LOG_HTTP</code> 3.0	ve vývoji ano	<code>1</code> = řádek na každý API požadavek i v produkci (bez těl)
<code>PUSH_ENDPOINT_HOSTS</code> 3.0	—	další povolené push služby (<code>.example.com</code> = i subdomény)
<code>VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code>	—	Web Push
<code>WEBAUTHN_RP_ID</code> , <code>WEBAUTHN_ORIGINS</code> , <code>PUBLIC_BASE_URL</code>	—	doména passkeys a povolené originy; bez nich produkce varuje
<code>STORAGE_DIR</code> , <code>STORAGE_MASTER_KEY</code> , <code>STORAGE_KEY_FILE</code>	—	úložiště a jeho master klíč
<code>ACCOUNTS_DIR</code> , <code>RELAY_RETENTION_DAYS</code>	—, 30	účty; jak dlouho čeká položka fronty bez vlastního TTL
<code>TURN_SERVER_URL</code> , <code>TURN_USERNAME</code> , <code>TURN_CREDENTIAL</code>	—	TURN pro striktní NAT
<code>LOG_EVENTS</code>	vypnuto	metadata událostí (<code>/api/admin/events</code>)
<code>ENABLE_AI</code> , <code>ENABLE_SPEECH</code> , <code>ENABLE_TELEPHONY</code> + klíče poskytovatelů	vypnuto	volitelné moduly

nginx

Konfigurace je v `deploy/nginx/m5cet.conf`. Pro 3.0 přibyl blok pro operátorské API (SSE potřebuje vypnutý buffering a dlouhý timeout):

```
location /api/admin/ {
    # allow 203.0.113.10; deny all; # doporučeno
    proxy_pass http://m5cet_app;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header Connection "";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffering off;
    proxy_cache off;
    proxy_read_timeout 3600s;
}
```

Upgrade z 2.x na 3.0

1. `git pull`, `npm ci`, `npm run build` (nebo `./update.sh`).
2. Do nginx přidat blok `/api/admin/`; `nginx -t && systemctl reload nginx`.
3. Běží-li hlavní služba jinde než na `127.0.0.1:$PORT`, nastavit admin službě `MAIN_URL`.
4. Restart `m5cet` a `m5cet-admin`. Při startu se staré schránky převedou do fronty (`queue.migrated` v auditu) a vznikne `signaling.secret`.
5. Otevřená okna nabídnou obnovení (nový build). Klient 2.x se s 3.0 nespojí — všichni obnoví.
6. Dřívější admin stránka ukládala token do `localStorage`; nová konzole ho při prvním otevření smaže. Zvažte výměnu `ADMIN_API_TOKEN`.

Ověření

```
npm run check           # TypeScript
npm test -- --no-file-parallelism # 700+ unit testů
npm run build
npm run test:e2e         # dva prohlížeče, účty a relay, konzole, lifecycle...
curl -s https://chat.example.com/api/health # {"ok":true,"protocol":2,"version":"3.0.0",...}
```

Bezpečnostní opatření serveru

HTTP

Helmet, CSP v produkci `script-src 'self' a object-src 'none'; Cache-Control: no-store` všude; limity **před** parsery těl (veřejné API 100 / 15 min, trezor 300, úložiště 1 200, admin 30 odmítnutí / 15 min); JSON do 256 KB (trezor 8 MB, úložiště 12 MB); chyby 500 bez interních detailů.

Logy

Řádek požadavku = metoda, cesta, stav, doba, velikost. **Nikdy těla** — odpovědi nesou session tokeny a šifrové texty (2.x je logovala). Surové tělo se drží jen pro webhooks poskytovatelů kvůli podpisům.

WebSocket

Origin, brána spojení, maxPayload 256 KB, validace rámců, limity tříd, vyhazování opakovaných narušitelů, heartbeat, zpětný tlak, vlastnictví přenosů, rozesílání jen ověřených polí.

Identita a soukromí

Pseudonymy účtů v místnosti, obecná odmítnutí relay, IP v monitoru zkrácené na /24 (/48), místnosti jako hash, audit komunikace jen na vyžádání, zapečetěné detaily v databázi.

SSRF

Server POSTuje jen na push endpointy známých služeb (https, bez portu a přihlašovacích údajů); příkaz ke stažení souboru přijme jen https adresu a klient se uživatele zeptá.

Ukončení

SIGTERM/SIGINT: sockety se zavřou (klienti se vrátí přes resume), účty se zapíší, databáze se zavřou a klíče vynulují, pak konec (nejpozději za 8 s).

Návrhy a roadmapa

Co bych v jakém modulu změnil nebo optimalizoval dál. Priorita P1 = nejvyšší přínos pro bezpečnost či spolehlivost, náročnost S / M / L.

Kryptografie

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Forward secrecy a odebrání člena — Sender Keys (řetěz klíčů na odesílatele, distribuce párově přes X25519) nebo MLS (RFC 9420) pro skupiny	Dnes je klíč místnosti statický: kdo heslo zná nebo získá, přečte i zachycený starší provoz, a člena nejde vyloučit bez změny hesla.	P1	L
Slepá ID místností — server dostane <code>HMAC(klíč místnosti, název)</code> místo názvu	Server by přestal znát názvy místností; pozvánky i join fungují stejně, protože heslo znají všichni členové.	P1	S
Párové šifrování soukromých zpráv (ECDH X25519 mezi identitami)	Zpráva pro vybrané příjemce je dnes šifrovaná klíčem místnosti; párový klíč ji omezí jen na ně i kryptograficky.	P2	M
Identita svázaná s účtem — veřejný klíč zařízení podepsaný v rámci účtu (passkey), seznam zařízení	Ověřený odznak „účet X“ napříč zařízeními místo TOFU podle jména.	P2	M
Argon2id (WASM) místo PBKDF2 pro heslo místnosti	Paměťově náročná KDF výrazně zdrazí útok GPU na slabá hesla; vyžaduje CSP <code>'wasm-unsafe-eval'</code> .	P2	S
Ověření QR kódem bezpečnostního čísla	Pohodlnější a spolehlivější než diktování 60 číslic.	P3	S

Protokol a signalizace

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Více instancí — stav místností přes Redis/NATS pub-sub, sticky sessions, fronta v Postgresu	Hub drží místnosti v paměti jednoho procesu; škálování a zero-downtime deploy potřebují sdílený stav.	P2	L
Binární rámce (CBOR) pro proxy souborů, případně WebTransport	Base64 v JSON přidává 33 % k datům a zátěž CPU na serveru i klientech.	P2	M
SFU pro skupinové hovory s E2EE (SFrame / insertable streams)	Mesh WebRTC se nad 4–5 účastníků neškáluje (upload roste s počtem lidí).	P3	L
TURN s krátkodobými přístupy (TURN REST API, HMAC)	Statické přihlašovací údaje k TURN jsou čitelné v klientovi a zneužitelné jako otevřený relay.	P1	S

Fronta a doručování

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Trvalá evidence účtenek (tabulka v globální DB)	Po restartu se dnes ztratí možnost poslat „přečteno“ ke zprávám relayovaným před ním (doručení přežije, je ve frontě).	P2	S
Neutrální text probuzení — jméno a místnost ukázat až po odemčení aplikace	Obsah push je šifrovaný pro prohlížeč (RFC 8291), push služba ho nepřečte; na zamčené obrazovce je ale dnes vidět jméno odesílatele a název místnosti.	P3	S
Metadata fronty do databáze uživatele (jméno odesílatele, místnost) — v globální DB jen směrovací minimum	Šifrový text je chráněný, ale kdo komu psal leží v globální DB čitelně pro server.	P2	M

Účty a úložiště

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Více passkeys na účet a obnovovací klíč	Ztráta jediného passkey dnes znamená ztrátu trezoru i databáze.	P1	M
Relace přežívající restart (hash tokenů v DB, obnovování, seznam zařízení s odhlášením)	Tokeny jsou jen v paměti: restart serveru odhlásí všechny a každý se musí znovu přihlásit passkey.	P2	S
Zálohy a kontrola integrity (SQLite backup API, <code>PRAGMA integrity_check</code> , plánovaný VACUUM)	Provozní jistota a obnova po havárii disku.	P2	S

Administrace a provoz

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Role a jmenné přístupy (auditor jen pro čtení / operátor), přihlášení do konzole passkey nebo OIDC	Jeden sdílený token neřekne, kdo co udělal, a nejde ho odebrat jednomu člověku.	P1	M
Neměnný audit — hash řetěz přes záznamy a podepsané kontrolní body	Pozdější úprava nebo smazání záznamu by byla odhalitelná.	P2	S
Prometheus <code>/metrics</code> , OpenTelemetry, pravidla alertů (např. víc než N bezpečnostních varování za minutu)	Napojení na existující monitoring a upozornění mimo otevřenou konzoli.	P2	S

Klient

NÁVRH	PROČ	PRIORITA	NÁROČNOST
Rozdělit <code>App.tsx</code> (4 000+ řádků) — hook spojení a signalizace, správce WebRTC peerů, úložiště zpráv (reducer), řadič přenosů; stavový automat spojení; error boundary	Testovatelnost a menší riziko regresí; dnes je většina logiky testovaná jen end-to-end.	P1	L
Kryptografie ve Web Workeru	PBKDF2 se 600 000 iteracemi zablokuje hlavní vlákno až na sekundu na slabším telefonu.	P2	S
Rozdělení bundlu (hlavní chunk 635 KB) a virtualizovaný seznam zpráv	Rychlejší první načtení a plynulost u dlouhých historií.	P2	M
Dokončit <code>i18n</code> — v <code>App.tsx</code> jsou natvrdo české a anglické řetězce	Konzistentní jazyk podle volby uživatele.	P3	S

Testování

- Fuzzing a property-based testy parseru rámců, metadat souborů a validace payloadů (fast-check).
- Zátěžové testy hubu (k6 / artillery): tisíce socketů, bouře relay, velké přenosy přes proxy — ověřit limity a zpětný tlak.
- E2E konzole v CI (test `admin-console.test.ts` už existuje) a test upgradu 2.x → 3.0 s daty.

Změny ve 3.0.0

Přidáno

Protokol v2 (validace, limity, brána, Origin, resume, pseudonymy, `auth`, odvolání, heartbeat, zpětný tlak), šifrování v2 (klíče podle účelu, AAD, zapečetěná signalizace, podpisy, TOFU, ověřené soubory), SQLite fronta s lease a dead-letter, nová konzole s živým provozem, auditem, uživateli, frontou, úložištěm a systémem, dokumentace HTML + PDF.

Opraveno

XSS přes MIME souboru, DoS přes metadata souboru, převzetí peer ID, podvržené účtenky a položky schránky, logování session tokenů, SSRF přes push endpoint, token admina v localStorage, příkazy z admin služby, které nikdy nedorazily, vlastní zprávy po obnovení označené jako cizí, 18 oprav úložiště.

Změněno

Veřejný status úložiště bez statistik ostatních, metadata endpointy jen s tokenem, kód zapečetěné zprávy 12 znaků, transfer log bez jmen souborů, historie serveru bez passkey šifrovaná v prohlížeči.

Nekompatibilní

Klienti 2.x se s 3.0 nespojí (nezapečetěná signalizace) a nepřečtou obálky v2. Po nasazení obnovit všechna okna; přidat blok nginx `/api/admin/`.

Podrobný seznam: `CHANGELOG.md`. Tato dokumentace odpovídá kódu verze 3.0.0.